GETTING STARTED

Copy

Environment Setup

To develop the Pintos projects, you'll need **two essential sets of tools**:

- **80×86 cross-compiler toolchain for 32-bit architecture** including a C compiler, assembler, linker, and debugger.
- **x86 emulator**: QEMU or Bochs

Option A (recommended): Docker

The easiest way to set up the development environment is Docker. **Your kind TAs have built a docker image for you in advance that contains all the toolchains to compile, run and debug Pintos.** This docker image has been tested on Mac (Intel chip), Mac (Apple M1 chip), Windows, and Linux.

Download

1. **First, you need to install docker on your computer.** Go to the docker download page↗ for help.
2. **Then pull the docker image and run it.** Just type the command below into your favorite shell (you can run `docker run --help` to find out what this command means in detail):

```
docker run -it pkuflyingpig/pintos bash
```

New this semester (Spring 2025):

We are rolling out an **experimental** docker image for ARM users as well, which you can run with:

```
docker run -it sevenchips/pintos-aarch64 bash
```

If you plan to setup Pintos via Docker Desktop on ARM machines (e.g. **Mac with Apple Mx chip**), this image should give you much better performance, as the default image comes in x86-64 and must be emulated on an ARM machine.

Your TAs have tested the standard solution from previous semesters and confirmed the basic functionalities of the new image. **We recommend ARM users try out this new option**, and you can always fall back to the default image simply by recompiling!

This image is about 3GB (it contains a full Ubuntu18.04), so it may take some time at its first run.

If everything goes well, you will enter a bash shell.

- Type `pwd` you will find **your home directory is under `/home/PKUOS`** .
- Type `ls` you will find that **there is a `toolchain` directory that contains all the dependencies**.

Now you own a tiny Ubuntu OS inside your host computer, and **you can shut it down easily by `Ctrl+d` or type `exit + enter`**. You can check that it has exited by running `docker ps -a`.

Boot Pintos

Now go back to your host machine, **git clone the Pintos skeleton code repository** by running:

```
git clone git@github.com:PKU-OS/pintos.git
```

> ***Note:***
>
> we have made some c*ustomization to the official Pintos distribution. So* ***you should be only getting the source code from the above channels****. In other words, do not download from other websites.*

Then run the docker image again but this time mount your `path/to/pintos` into the container.

```
docker run -it --rm --name pintos --mount
type=bind,source=absolute/path/to/pintos/on/your/host/machine,targ
et=/home/PKUOS/pintos pkuflyingpig/pintos bash
```

> ***Note:***
>
> ***Do not just copy and paste the command above!*** *At least you need to replace the absolute path to your pintos directory in the command!*

> **`Docker notes:`**
>
> - **`--rm:`** Tell docker to delete the container after running
> - **`--name pintos:`** Name the container as `pintos`, which will be helpful in the debugging part.

> You may use this command throughout this semester, and it is tedious to remember it and type it again and again. You can choose the way you like to avoid this. **e.g. You can use the `alias` Linux utility to save this command as `pintos-up` for example.**

Now when you `ls`, you will find there is a new directory called `pintos` under your home directory in the container, **it is shared by the container and your host computer i.e. any changes in one will be synchronized to the other instantaneously**.

Now here comes the exciting part, type the following commands in turn:

```
cd pintos/src/threads/
make
cd build
pintos --
```

Bomb! The last command will trigger Qemu to simulate a 32-bit x86 machine and boot your Pintos on it. If you see something like:

```
Pintos hda1
Loading............
Kernel command line:
Pintos booting with 3,968 kB RAM...
367 pages available in kernel pool.
367 pages available in user pool.
Calibrating timer...  32,716,800 loops/s.
Boot complete.
```

Your Pintos has been booted successfully, congratulations :)

You can shut down the Qemu by `Ctrl+a+x` **or** `Ctrl+c` if the previous one does not work.

By far Pintos will exit immediately after booting, so you may not see much useful information in the output. Don't be disappointed.

If you want to understand the internal details of the docker image, you can look through the dockerfile ↗ on Github.

What's the magic?

Now Let's conclude what you have done.

- First, You used docker to run a Ubuntu container that functions as a full-edged Linux OS inside your host OS.
- Then you used Qemu to simulate a 32-bit x86 computer inside your container.

- Finally, you boot a tiny toy OS -- Pintos on the computer which Qemu simulates.

Wow, ***virtualization is amazing***, right?

Throughout this semester, **you can modify your Pintos source code in your host machine with your favorite IDE but compile/run/debug/test your Pintos in the container thanks to the mounting technique**. You can leave the container running when you are modifying Pintos source code in you host computer, because your modification will be visible in the container immediately.

Option B: Virtual Machine

> This method is not fully tested by your TAs, so we may not provide much help if you encounter some strange errors.

If you would like to use a VM for development, there is a VirtualBox↗ VM provided by Johns Hopkins Univerisity that runs Ubuntu 18.04 with the necessary toolchain installed. You can download the image here↗. The image is large (2.5 GB), so the download can take a while. The md5sum for the VM image is `69c89938d4b768bdcca4362fd39f06e4`. The initial login password is `jhucs318`.

When you enter the VM successfully, you can follow the instruction here to boot Pintos. You should ignore all the instructions related to Docker.

Previous
Welcome to Pintos

Next
Build and Run

Last updated 1 year ago